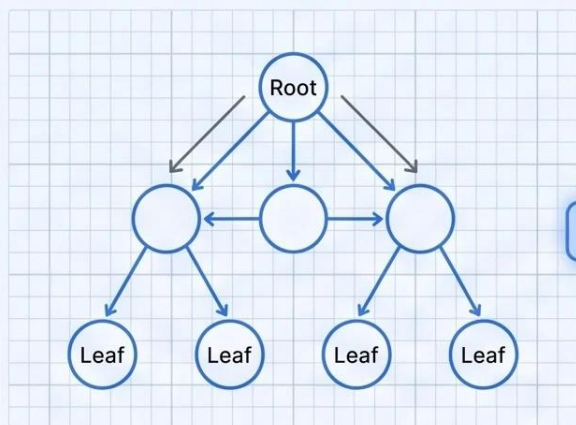


# 트리 자료구조의 기초 개념 이해

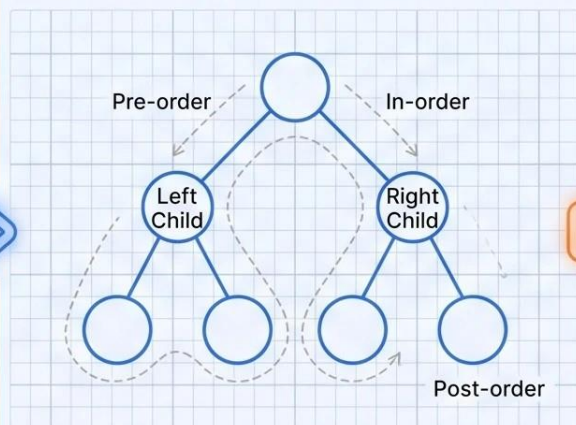
트리부터 이진 트리, 힙까지의 단계적 학습 흐름

## 1단계: 트리의 구조와 기본 개념



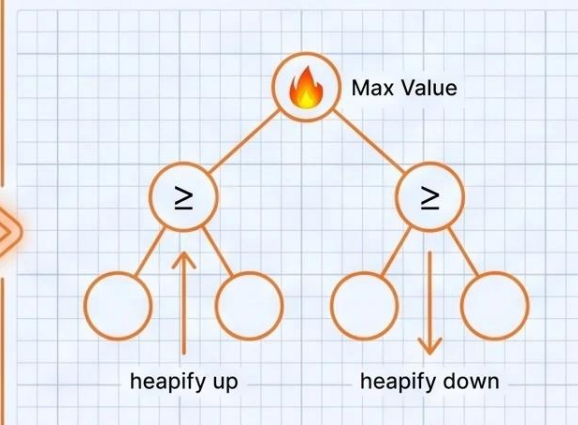
노드, 엣지, 루트, 리프, 레벨, 높이 등  
기본 용어 및 개념 이해

## 2단계: 이진 트리의 표현과 순회 방식



최대 2개의 자식 노드 구조,  
배열/링크드 리스트 표현,  
순회(전위, 중위, 후위) 방법

## 3단계: 최대 힙의 구조와 연산 원리



완전 이진 트리 기반,  
최대 힙 속성(부모  $\geq$  자식),  
삽입/삭제 연산 및 응용

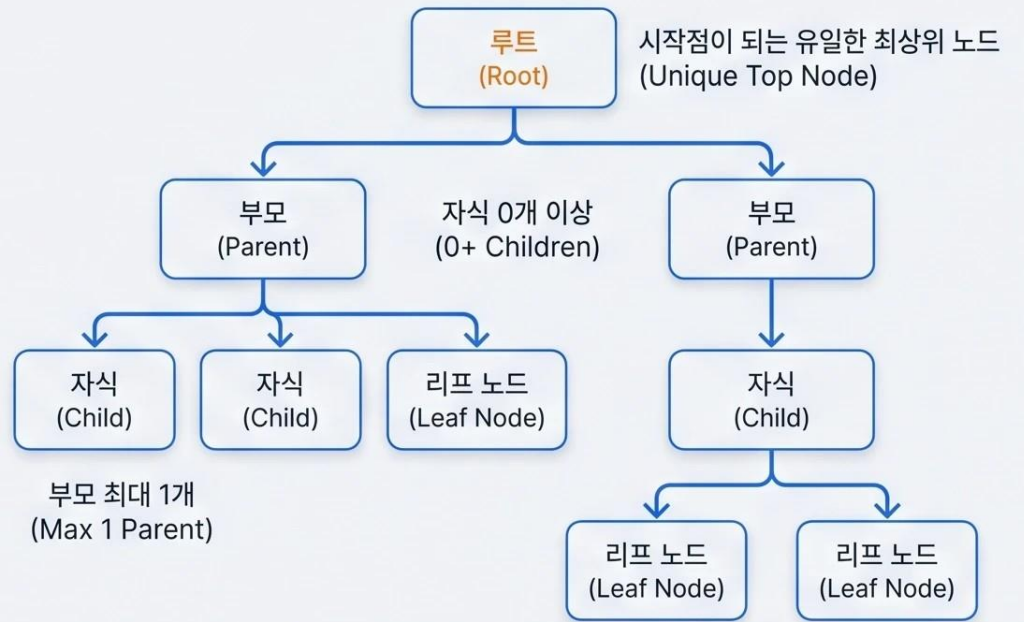
**목표:** 트리 기반 자료구조의 단계적 이해 및 응용 능력 배양

# 트리 자료구조 (Tree Data Structure) =not linear

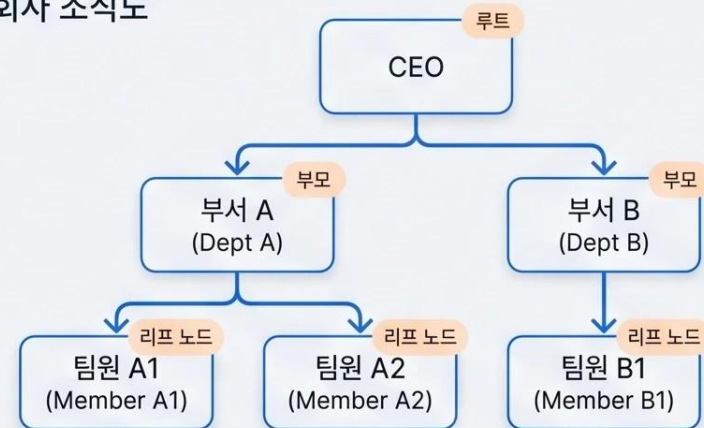
## 선형 구조 (Linear Structure)



## 트리 구조 (Tree Structure)



### 예시: 회사 조직도





# 선형 구조 vs 트리: 계층과 분기의 이해

## ■ 선형 구조

1차원 순서 구조: 원소가 일렬로 나열

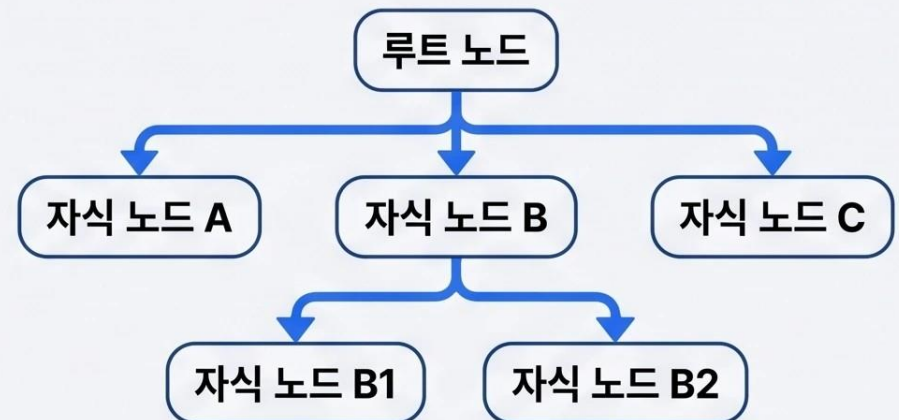


이전 ↔ 다음 관계만 존재  
(앞/뒤 접근·삽입·삭제)

순서

## ■ 트리

계층적 비선형 구조: 여러 방향으로 분기



한 노드 → 여러 자식으로 확장  
(부모-자식 관계)

분기/계층

예시: 폴더 구조 (계층적 포함 관계)



# 트리 구조의 기본 단위: 노드(Node)와 루트(Root)

## 노드 vs 루트 (NODE vs ROOT)



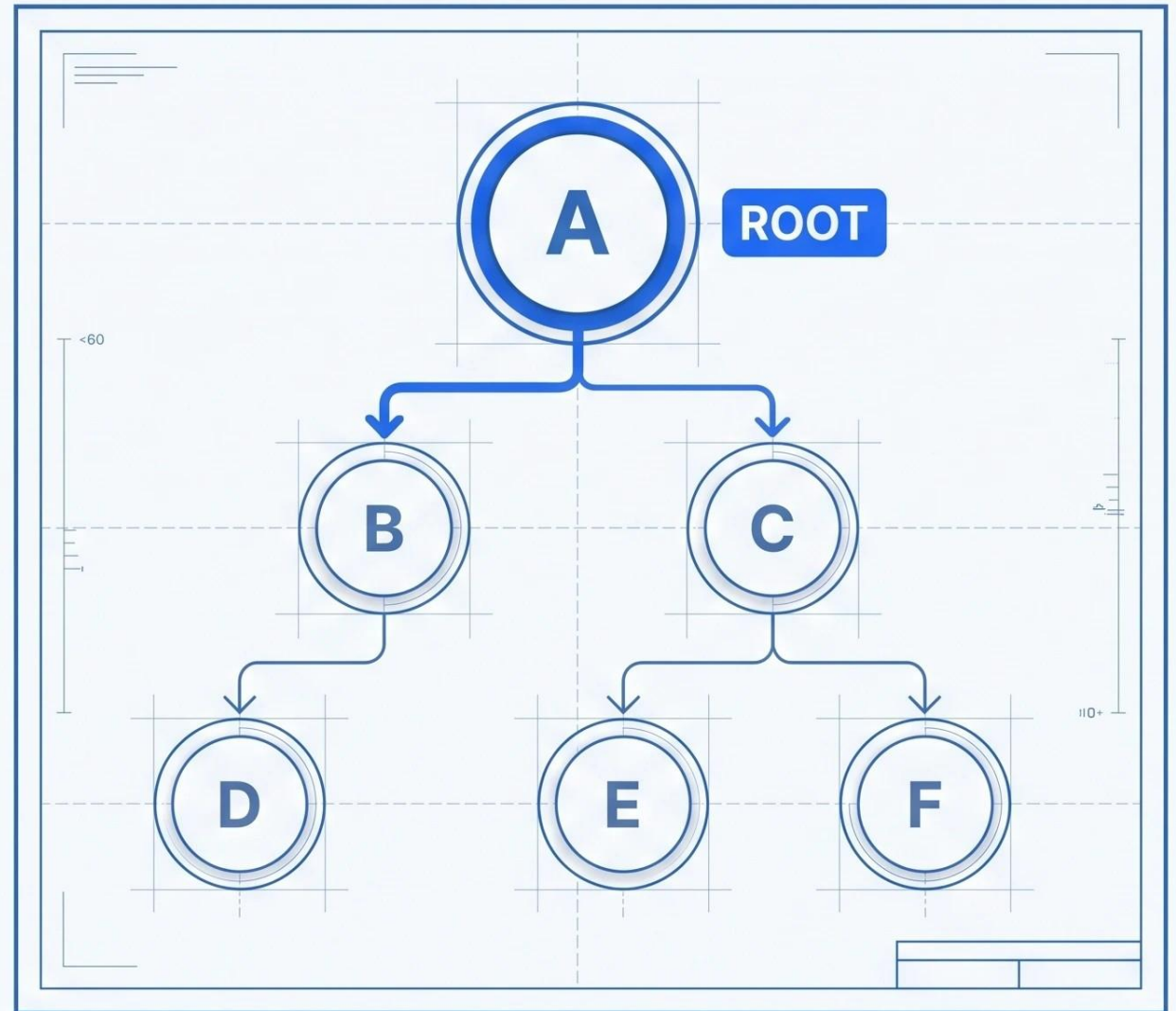
### 노드 (Node)

데이터를 담고 다른 노드와 연결되는 트리의 기본 단위



### 루트 (Root)

부모가 없는 유일한 노드



**예시:** A는 루트(Root), B·C·D·E·F는 모두 노드(Node)로서 루트에서 시작해 확장된 구조입니다.



# 서브트리 및 노드 분류 정의와 예시



## 서브트리 (Subtree)

그림에서 서브트리는 3개임(A포함, D는 아님)

특정 노드 + 그 노드의 모든 자손으로 이루어진 트리



## 단말노드 (Leaf)

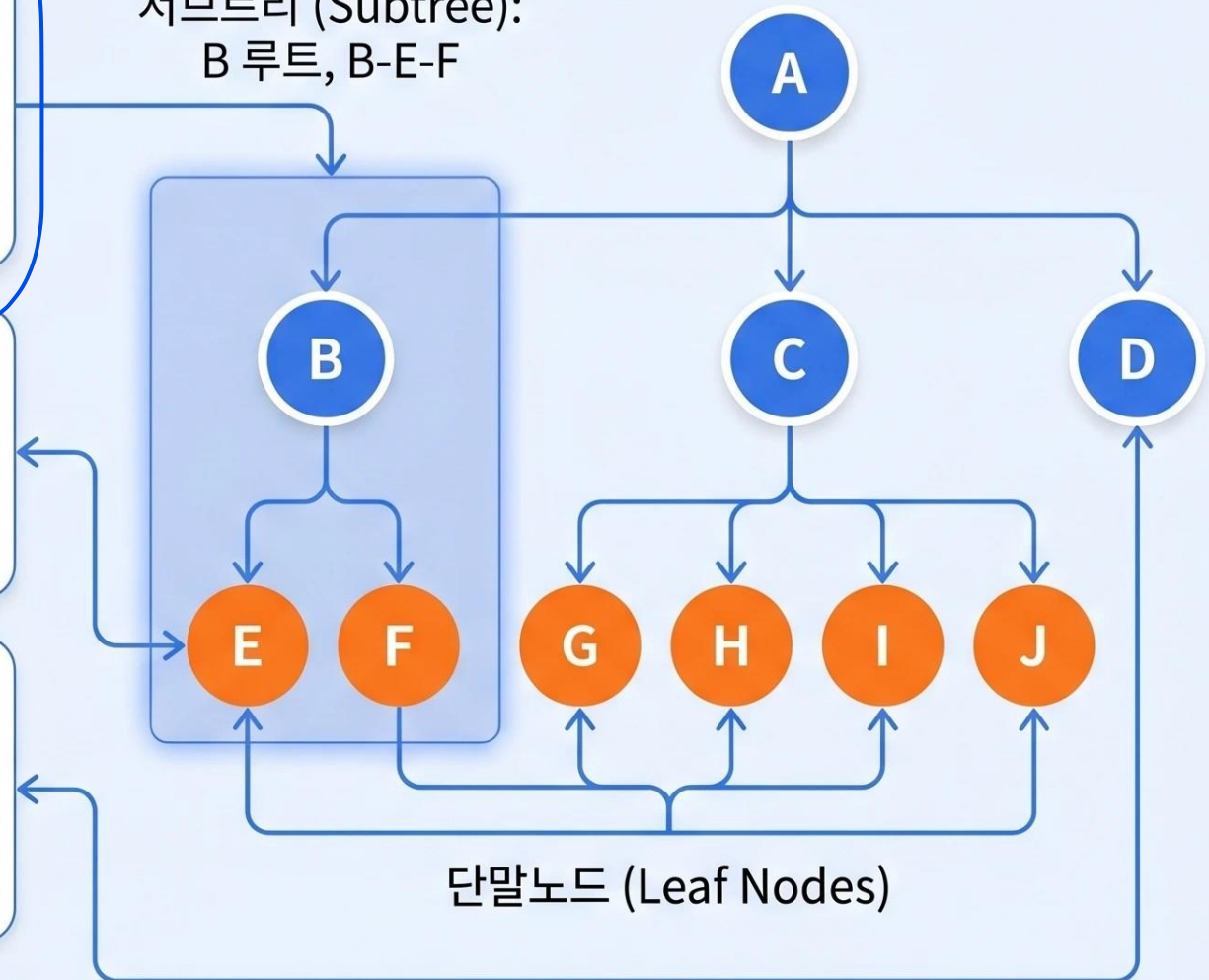
자식이 하나도 없는 노드



## 비단말노드 (Non-terminal)

하나 이상의 자식을 가진 노드

서브트리 (Subtree):  
B 루트, B-E-F



트리는 부분 구조도 독립적인 트리로 유지됨  
→ 재귀적 처리 가능

비단말노드 (Non-terminal Nodes)

# 트리(Tree)의 레벨, 높이, 차수



## 레벨(level):

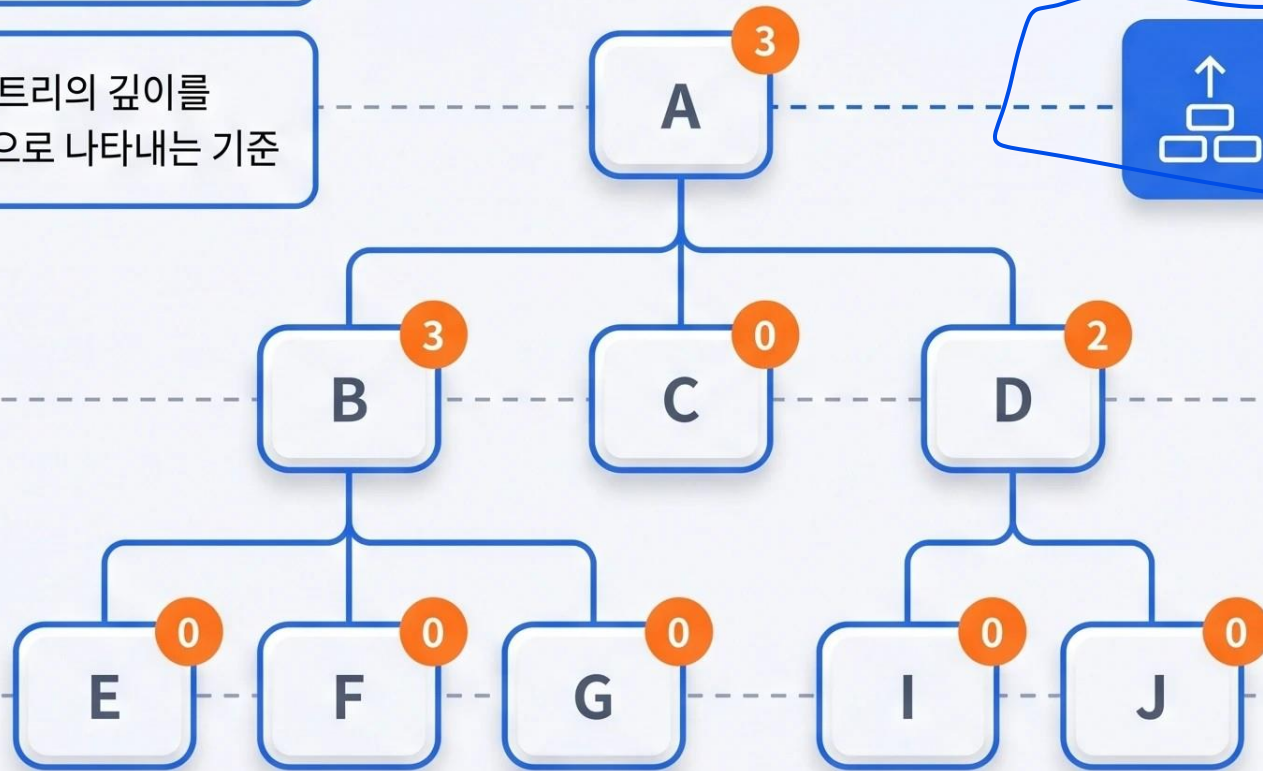
루트에서 아래로 내려가며  
각 층에 번호를 매긴 값

Level 1

레벨은 트리의 깊이를  
단계적으로 나타내는 기준

Level 2

Level 3



최대차수 = 3



## 높이(height):

트리에서 가장 큰 레벨 값



**Height = 3**

(트리 전체의 최대 깊이)



## 차수(degree):

각 노드가  
가진 자식 노드의 개수



# 트리 용어 (루트, 부모, 자식, 조상, 자손)

## 루트 (Root)

최상단 노드,  
부모가 없는 노드

## 부모-자식 (Parent-Child)

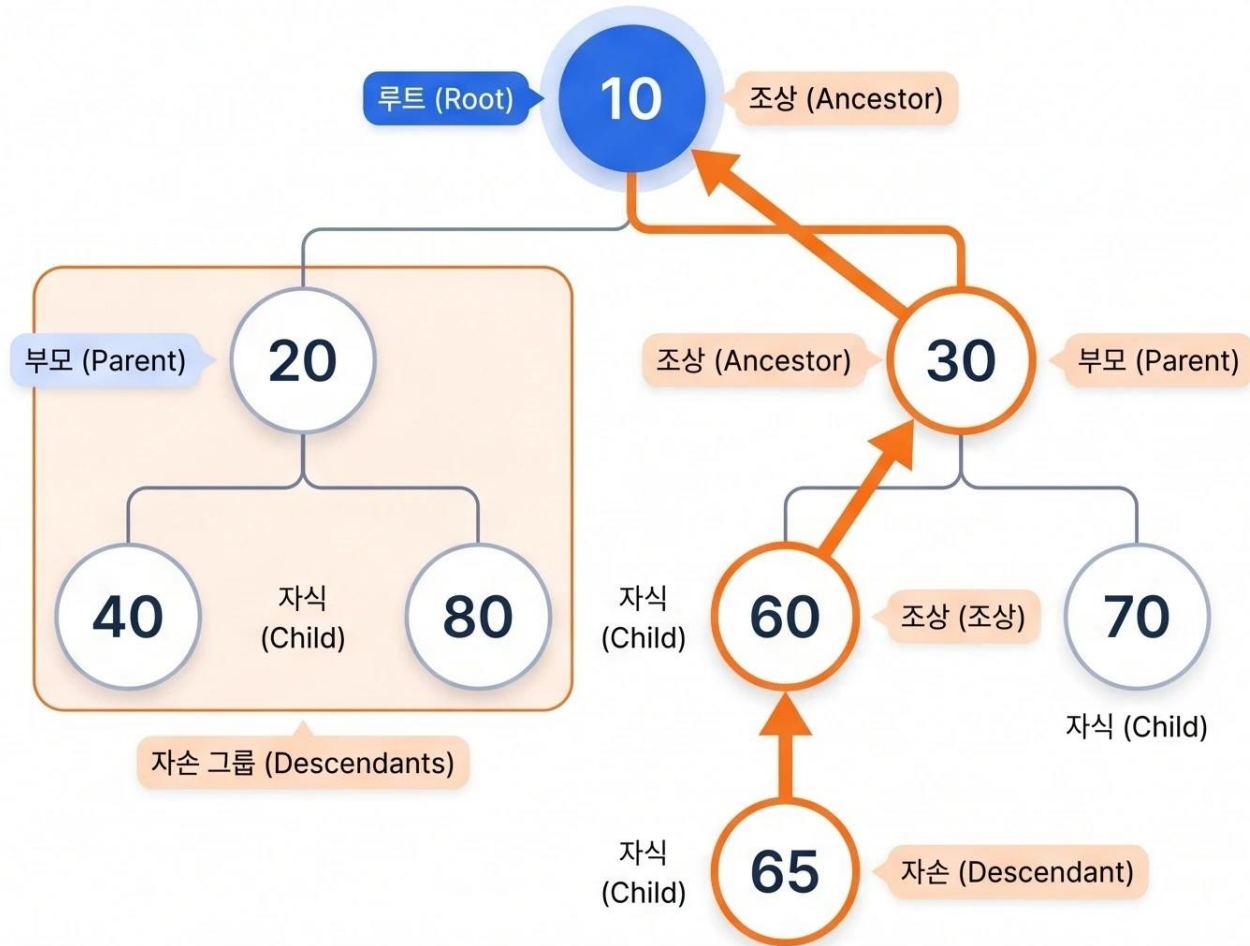
직접 연결된  
상하 관계 (1단계)

## 조상 (Ancestor)

자신을 제외한  
모든 상위 노드 경로

## 자손 (Descendant)

자신을 제외한  
모든 하위 노드 경로



### 조상 경로 흐름 (65의 조상)

65 자손 → 60 조상 → 30 조상 → 10 루트/조상

다단계 관계

### 자손 그룹 (20의 자손)

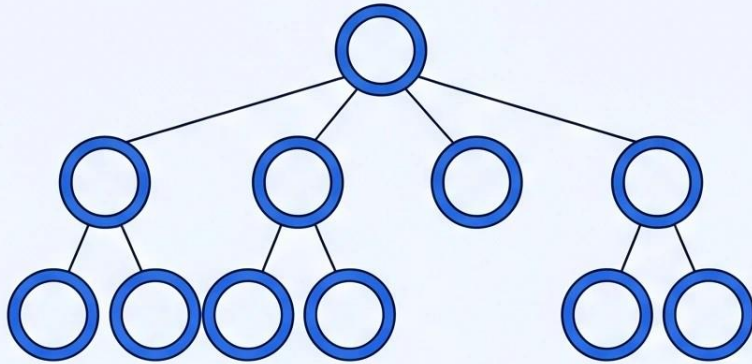
20 부모 → 40 자손 → 80 자손

동일 색 그룹 묶음

# 트리와 이진 트리의 관계 및 응용

이건 트리냐? -> 강 노드임

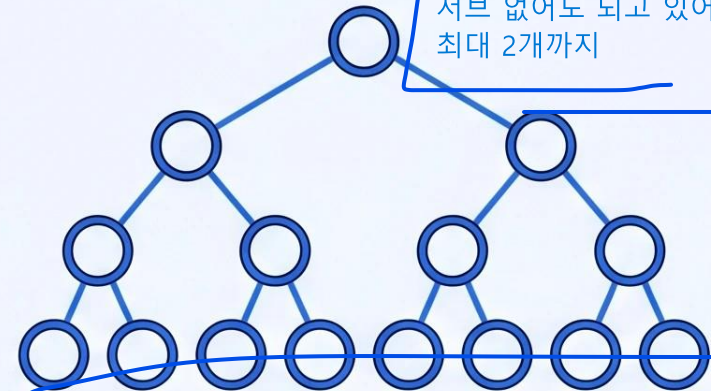
## 트리 (Tree)



자식 수에 제한이 없는 계층 구조  
일반적인 계층적 데이터 표현

특수한 형태

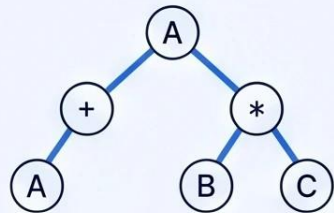
## 이진 트리 (Binary Tree)



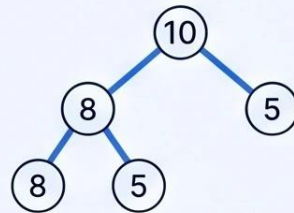
서브 없어도 되고 있어도 되는데,  
최대 2개까지

각 노드가 최대 2개의 서브트리 (왼쪽, 오른쪽)  
구조가 단순하여 메모리 표현과 알고리즘 설계 용이

## 응용 구조 예시



수식 트리 (Expression Tree)

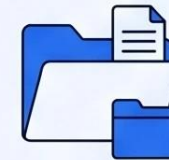


힙 (Heap)

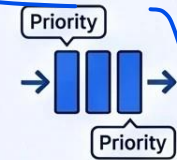
## 활용 사례



컴파일러  
(Compiler)



파일 시스템  
(File System)

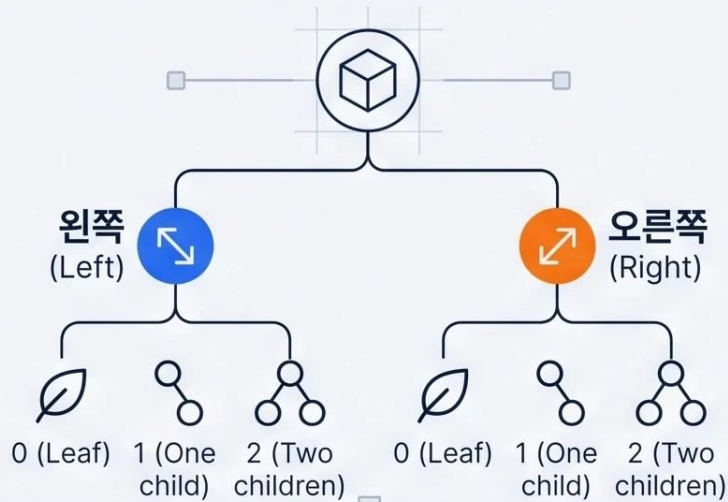


우선순위 큐  
(Priority Queue)



# 이진 트리 (Binary Tree) 정의 및 특징

## 이진 트리 정의

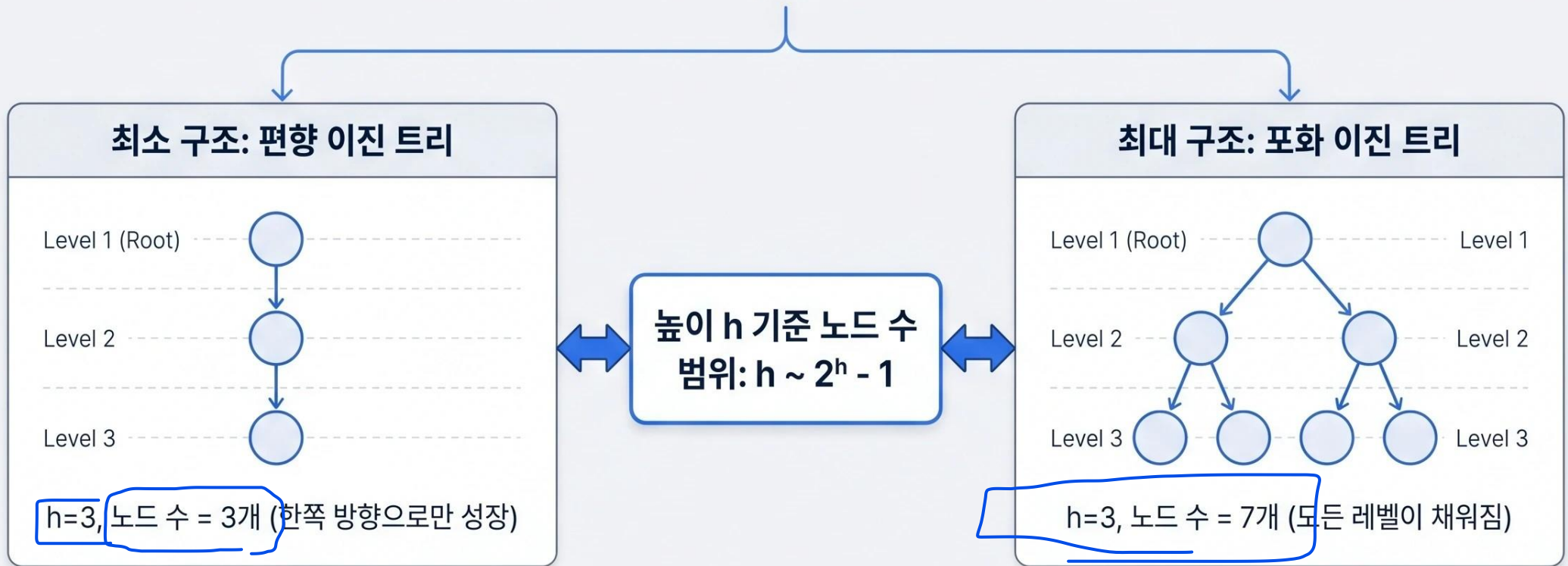


## 순서의 중요성



|   |       |                                   |
|---|-------|-----------------------------------|
| 1 | 자식 수  | 0    1    2                       |
| 2 | 최대 차수 | $\leq 2$                          |
| 3 | 구조 특징 | 좌/우 구분 강조<br><b>왼쪽</b> <b>오른쪽</b> |

# 이진 트리 높이(h)와 노드 수 관계



높이 h일 때 최소 노드 수 =  $h$ 개

높이 h일 때 최대 노드 수 =  $2^h - 1$ 개

예시 및 활용 (h=3 비교)

$2^h - 1$

| 구조 비교                          | 노드 수    |
|--------------------------------|---------|
| 편향 트리 (Skewed Tree)            | 3개 (최소) |
| 포화 트리 (Full Tree)              | 7개 (최대) |
| 활용: 탐색 깊이 및 알고리즘 시간 복잡도 분석에 사용 |         |

$2^h - 1$



# 이진 트리 높이의 범위와 노드 수의 관계 (h와 n의 관계)

노드 수  $n$

높이  $h$  정의: 루트부터 가장 깊은 노드까지의 경로 길이

예시 :  $n=7 \rightarrow$  최대  $h=7$ , 최소  $h=\log_2(8)=3$

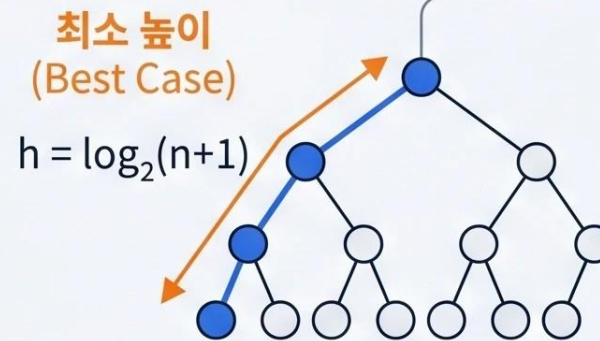
$\log_2(n+1)$

$h$  범위

$n$



편향 이진 트리: 한 줄로 이어진 구조



균형 이진 트리: 레벨이 거의 꽉 찬 구조

구조

특징

높이

편향

한 줄

$n$  (최대/Max)

균형

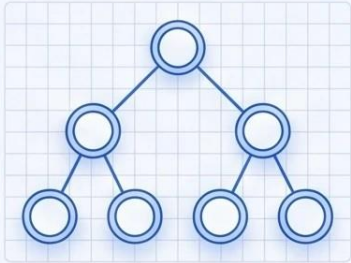
레벨 채움

$\log_2(n+1)$  (최소/Min)

균형  $\uparrow \rightarrow$  높이  $\downarrow$  / 편향  $\uparrow \rightarrow$  높이  $\uparrow$

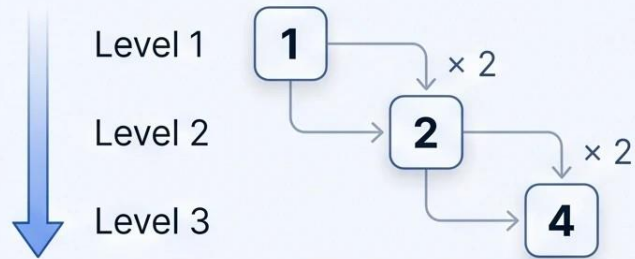
# 포화 이진 트리

## 정의



✓ 모든 레벨이 꽉 참

## 수식 구조



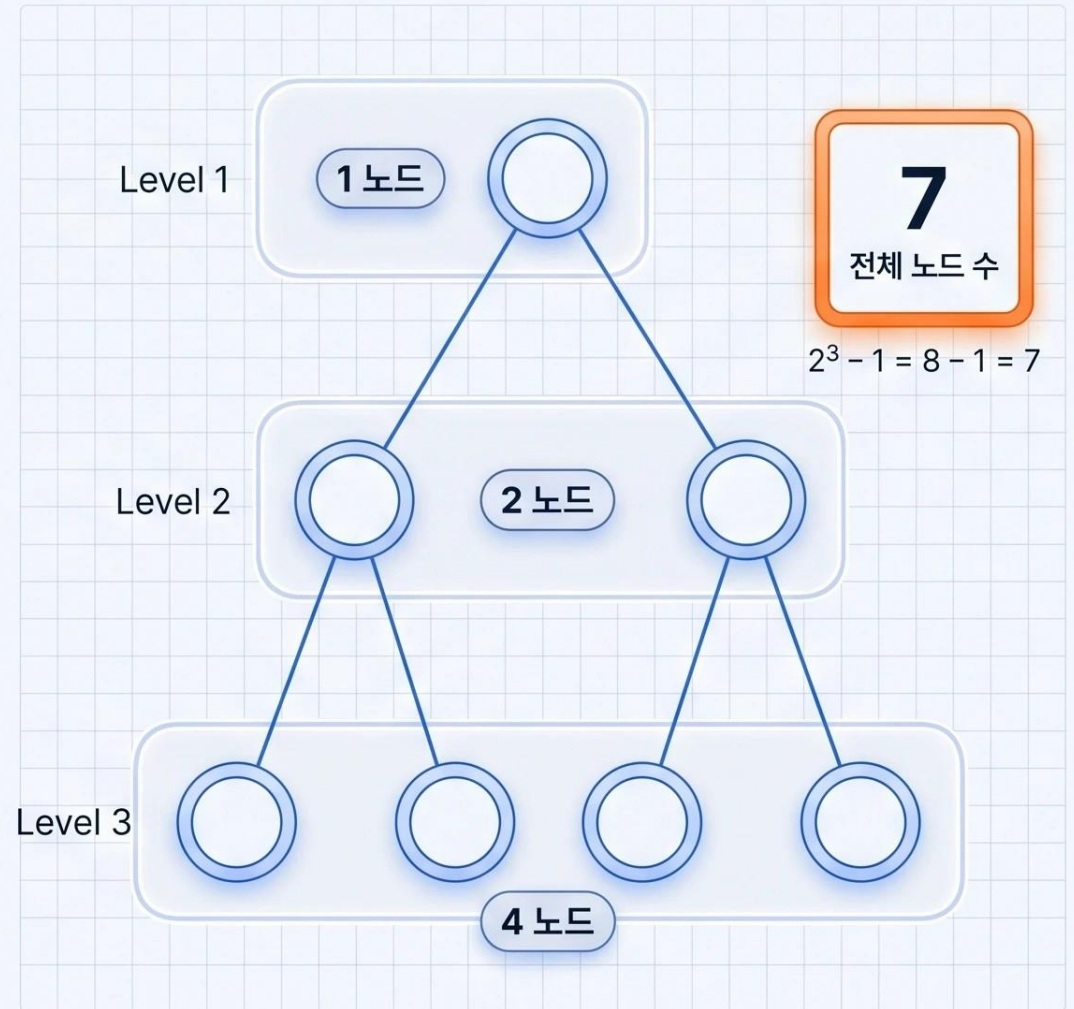
**전체 노드 수:  $2^k - 1$**

$k = \text{높이}$

레벨별 노드 수:  $2^{(\text{레벨}-1)}$

특징: 최소 높이 + 최대 노드 밀도 (이상적 구조)

## 예시: 높이 3



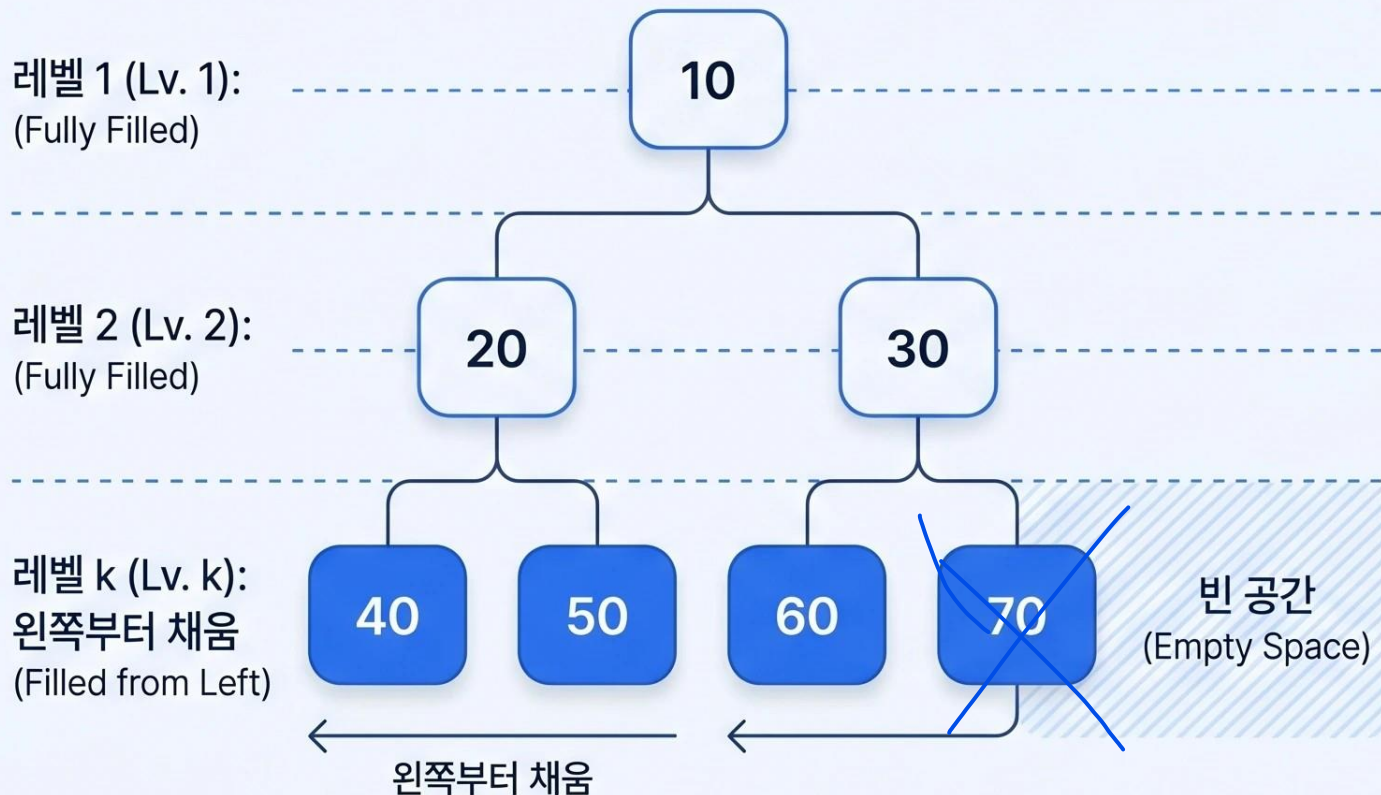


# 완전 이진 트리 (Complete Binary Tree)

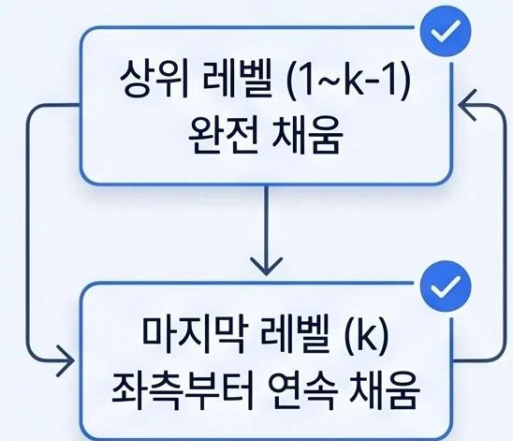
**정의 (Definition):** 레벨 1~k-1은 모두 채워지고, 마지막 레벨은 왼쪽부터 연속 채워진 이진 트리. 끝까지 안 채워짐

- 마지막 레벨에서는 오른쪽 끝에만 빈 칸 허용 (중간 공백 불가)
- 힙(heap) 구현의 기본 구조로 사용됨

2<sup>k-1</sup>



## 조건 구조



## 잘못된 예시 불완전 (Incomplete)

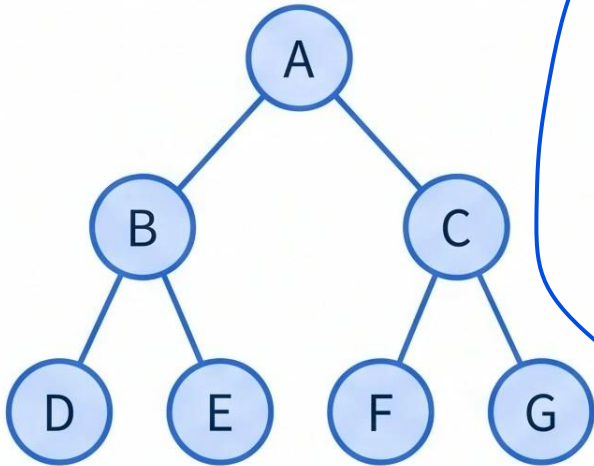




# 포화 이진 트리 vs 완전 이진 트리: 정의와 관계

## 포화 이진 트리

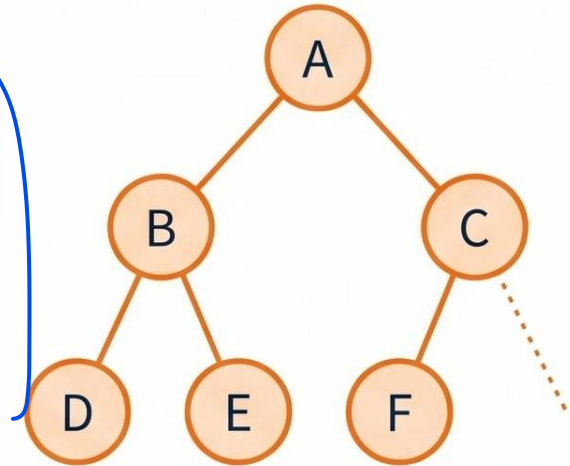
정의: 모든 레벨이 빈 자리 없이  
꽉 찬 트리



예시 구조 (포화이자 완전):  
A-B,C / B-D,E / C-F,G

## 완전 이진 트리

정의: 마지막 레벨만 왼쪽부터  
순서대로 채워진 트리



예시 구조 (완전하지만 포화 아님):  
A-B,C / B-D,E / C-F  
(C의 오른쪽 자식 비어있음)

완전 이진 트리

포화 이진 트리

관계: 포화  $\subset$  완전

모든 포화 이진 트리는  
완전 이진 트리지만,  
그 반대는 아님

# 이진 트리 표현 방식

## 이진 트리 표현 방식

### 배열 표현

#### 정의

노드 번호를 배열 인덱스로 사용하는 방식

#### 특징

완전 이진 트리에서 메모리 효율적, 인덱스로 탐색 빠름

#### 예시

높이 3 완전 이진 트리  
→ 배열 인덱스 1~7에 저장

### 링크 표현

#### 정의

포인터로 부모가 자식을 가리키는 방식

#### 특징

구조 변경 유연, 편향 트리에서 메모리 낭비 감소

#### 예시

링크 방식 → 각 노드가 left, right 포인터로 연결



# 이진 트리의 배열 표현법 (Array Representation)

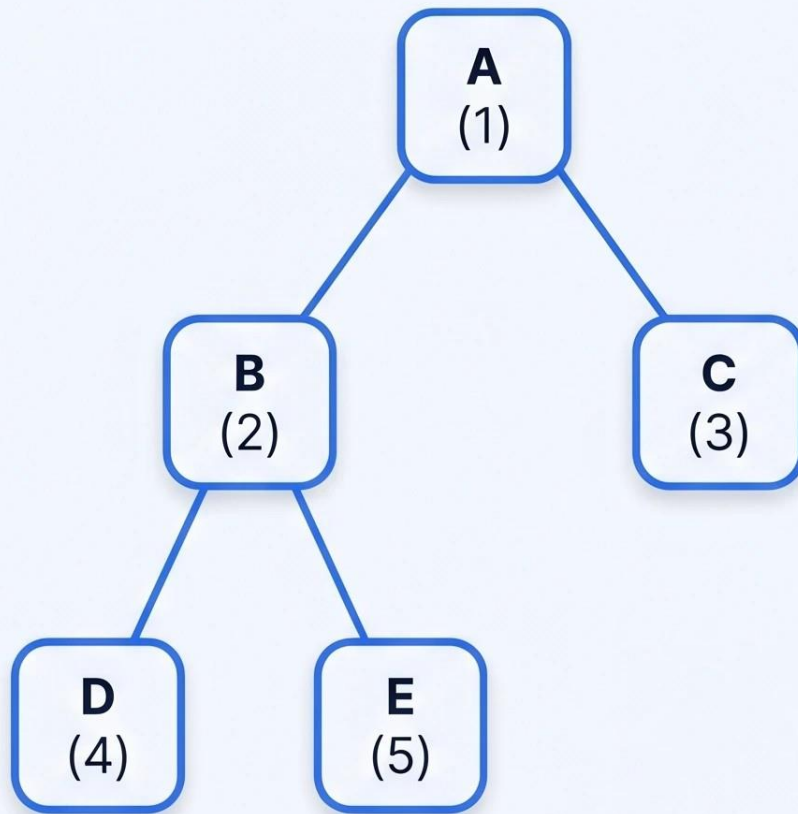
**정의:** 모든 이진 트리를 포화 이진 트리로 가정하고,  
노드 번호를 배열 인덱스로 사용해 데이터를 저장

## 인덱스 규칙

부모 ( $i$ )  $\rightarrow$  왼쪽 자식 ( $2i$ ), 오른쪽 자식 ( $2i+1$ )

$h \rightarrow \{$

# 배열 기반 이진 트리 인덱스 계산 규칙



## i 기준 관계식

부모

$i / 2$  (정수 나눗셈)

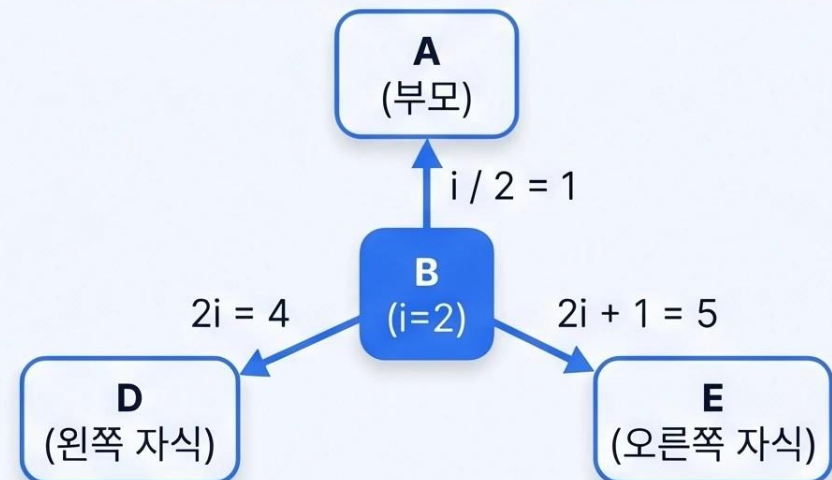
왼쪽 자식

$2i$

오른쪽 자식

$2i + 1$

## 적용 예시: $i = 2$ (B)



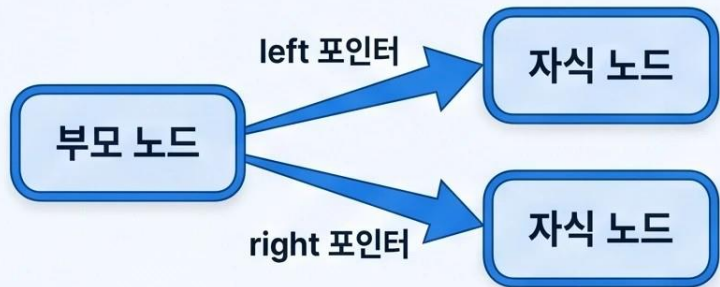
**설명:** 완전 이진 트리에서 위치 계산을 단순화하는 핵심 규칙



# 링크 표현법 (Linked Representation)

트리 구조의 연결 기반 데이터 표현 및 관리 (Connected Data Representation & Management in Tree Structures)

## 개념 다이어그램



**정의:** 포인터 → 부모 노드가 자식 노드를 가리키는 연결 방식  
**특징:** 동적 연결 → 메모리 효율 + 삽입/삭제 용이

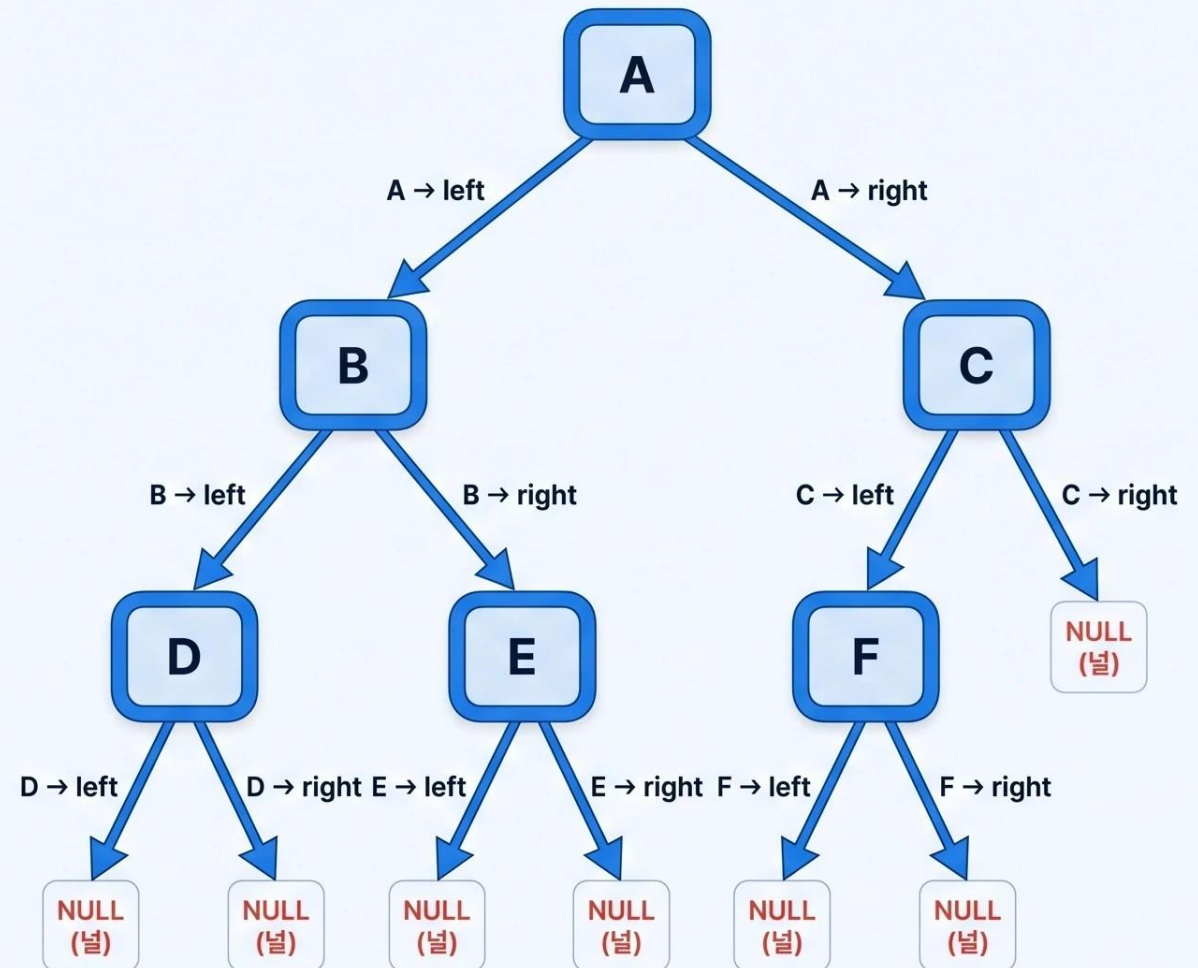
## 노드 구조



## C 구조체 코드

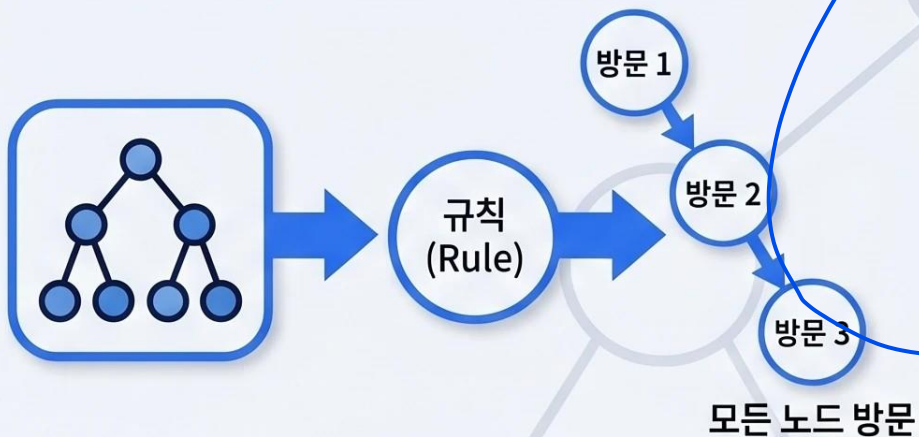
```
code card
struct TreeNode {
    int data;
    TreeNode *left;
    TreeNode *right;
};
```

## 예시 구조



# 이진 트리 순회 (Binary Tree Traversal): 개념 및 핵심

## 정의 (Definition)



순회는 트리의 모든 노드를  
일정한 규칙에 따라 하나씩 방문하는 것

## 이진 트리의 순회 종류

|                  | 방식  | 특징 키워드                                |
|------------------|-----|---------------------------------------|
| 전위 (Pre-order)   | DFS | 루트-좌-우 (Root-L-R) <small>스택방식</small> |
| 중위 (In-order)    | DFS | 좌-루트-우 (L-Root-R)                     |
| 후위 (Post-order)  | DFS | 좌-우-루트 (L-R-Root)                     |
| 레벨 (Level-order) | BFS | 층별 순회 (Layer-by-Layer)                |

## 순회의 목적



읽기  
(Read)



합계  
(Sum)

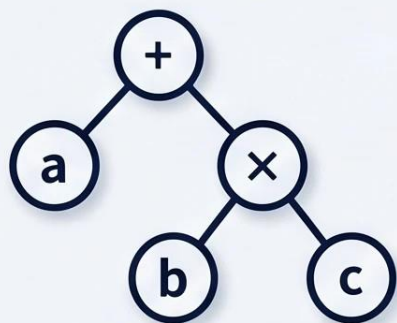


검색  
(Search)



출력  
(Output)

## 예시: 수식 트리와 중위 순회



중위 순회 적용

$(a + (b \times c))$

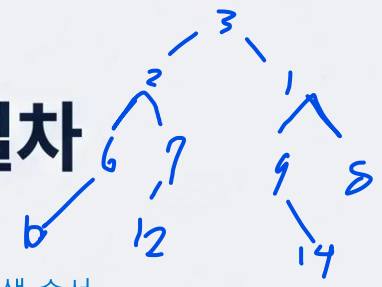
결과: 중위 표기식

$a + b \times c$



# 전위 순회(Preorder Traversal) 규칙 및 절차

VLR 규칙: 루트 → 왼쪽 → 오른쪽

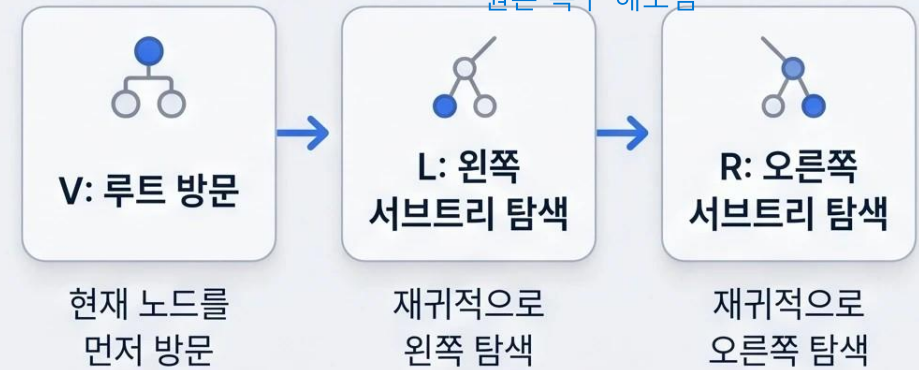


탐색 순서

3, 2, 6, 10, 7, 12, 1, 9, 14, 8

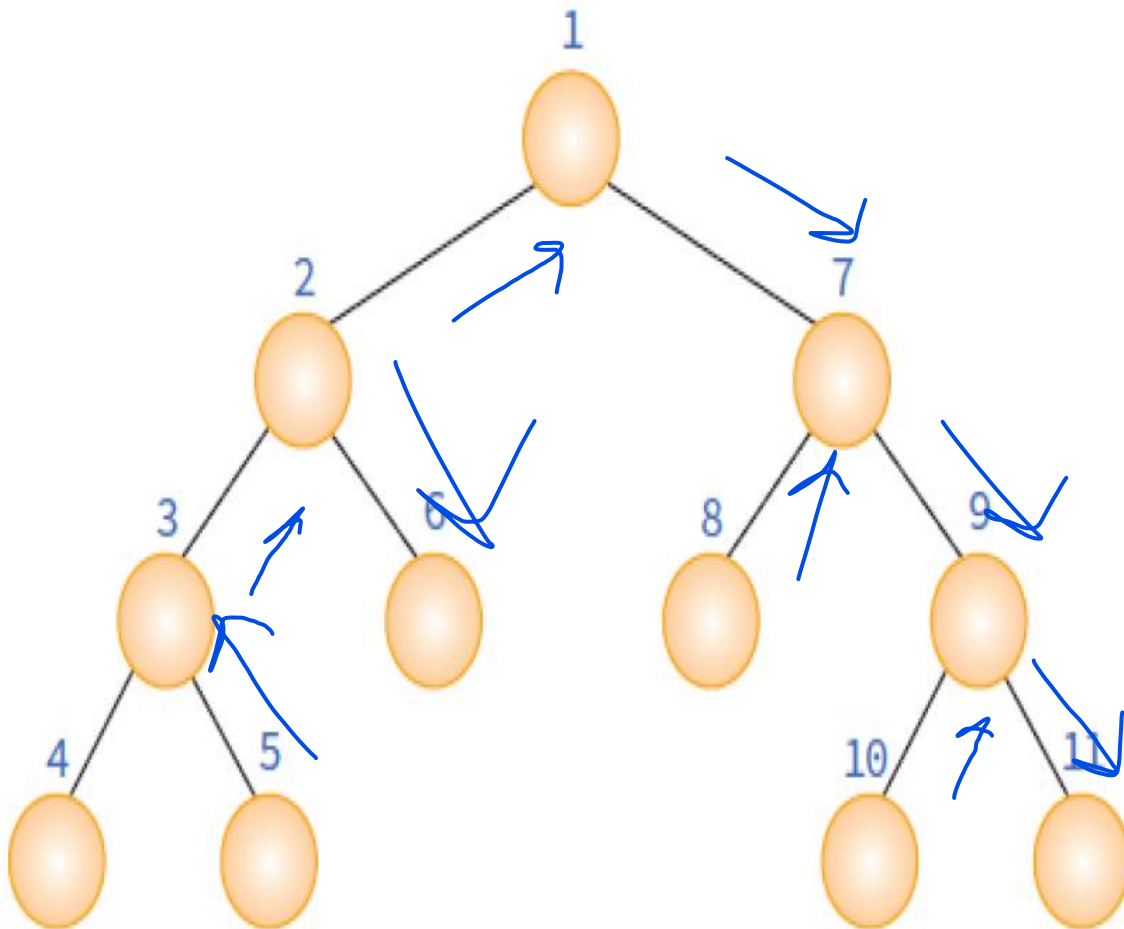
원본 복구 해보셈

## V→L→R 규칙 순서도



## 의사코드 (Pseudocode)

```
if (x ≠ NULL)
    print(x)           // V: 현재 노드 출력
    preorder(x->LEFT)  // L: 왼쪽 재귀 호출
    preorder(x->RIGHT) // R: 오른쪽 재귀 호출
```



전위 순회 결과 (Example Result):

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11

활용: 트리 구조를 선형 형태로 표현하거나 복사할 때 유용



# 중위 순회 (In-order Traversal) 규칙: LVR 흐름

왼쪽 → 루트 → 오른쪽 순으로 방문



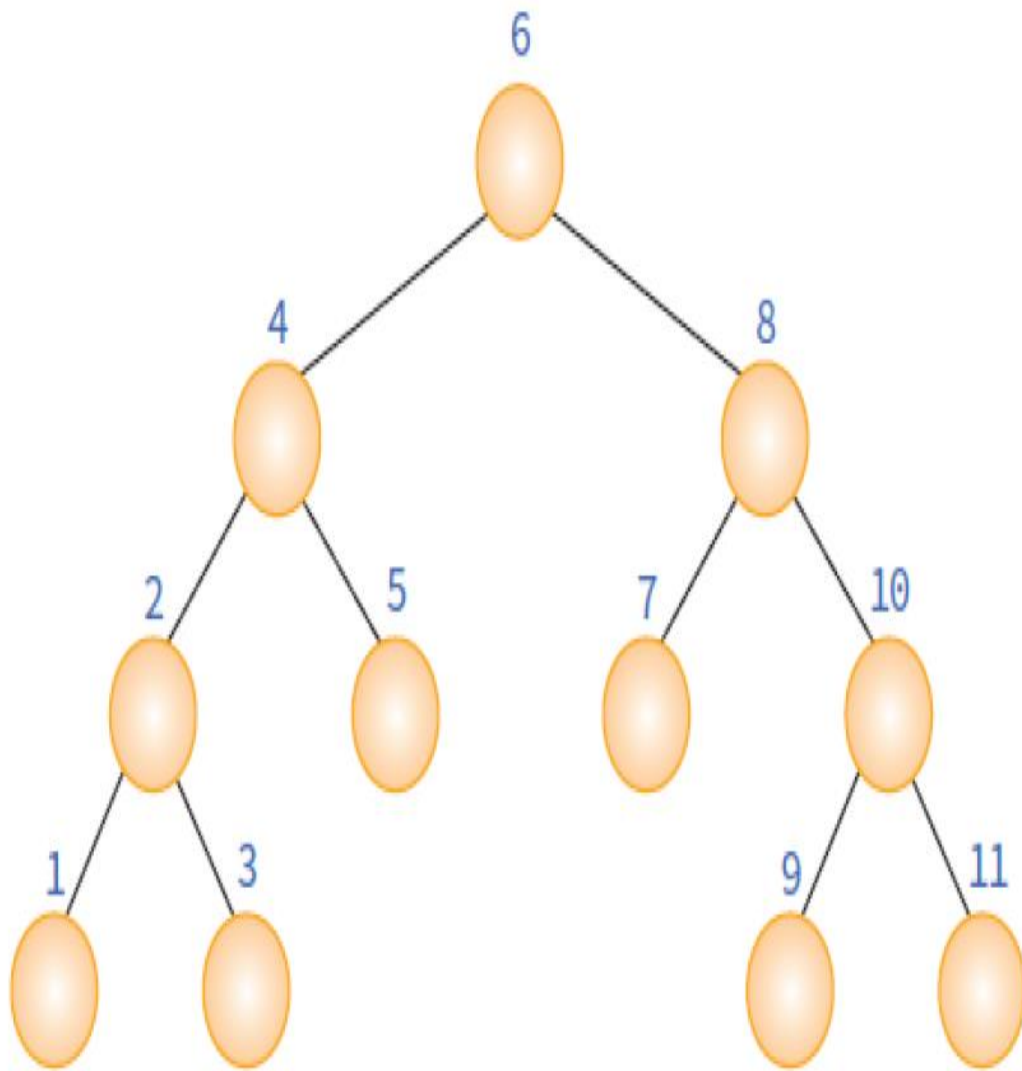
L (왼쪽 서브트리)



V (루트 방문)



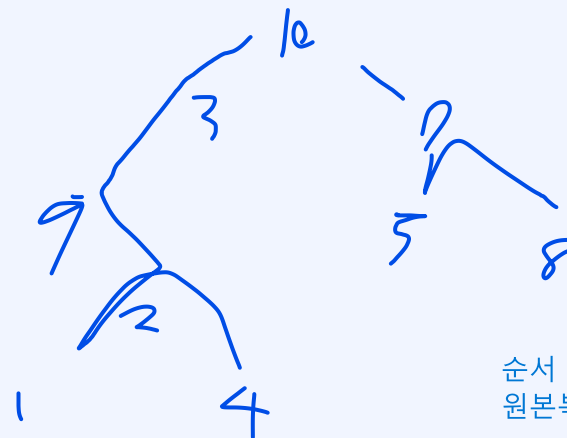
R (오른쪽 서브트리)



특징: BST 결과 (오름차순 정렬)

|   |   |   |   |   |   |   |   |   |    |    |
|---|---|---|---|---|---|---|---|---|----|----|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|----|----|

이진 탐색 트리(BST) 중위 순회 시,  
결과는 항상 오름차순으로 정렬됨



순서 : 9, 1, 2, 4, 3, 10, 5, 7, 8  
원본복구도 해보셈

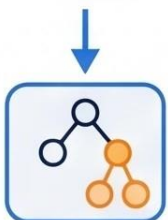
# 후위 순회 (Post-order Traversal): LRV 규칙과 활용

## (The LRV Rule and Its Applications)

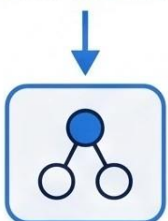
### 정의 및 절차 (Definition & Procedure)



L: 왼쪽 서브트리 후위 순회  
(Left Subtree Post-order)

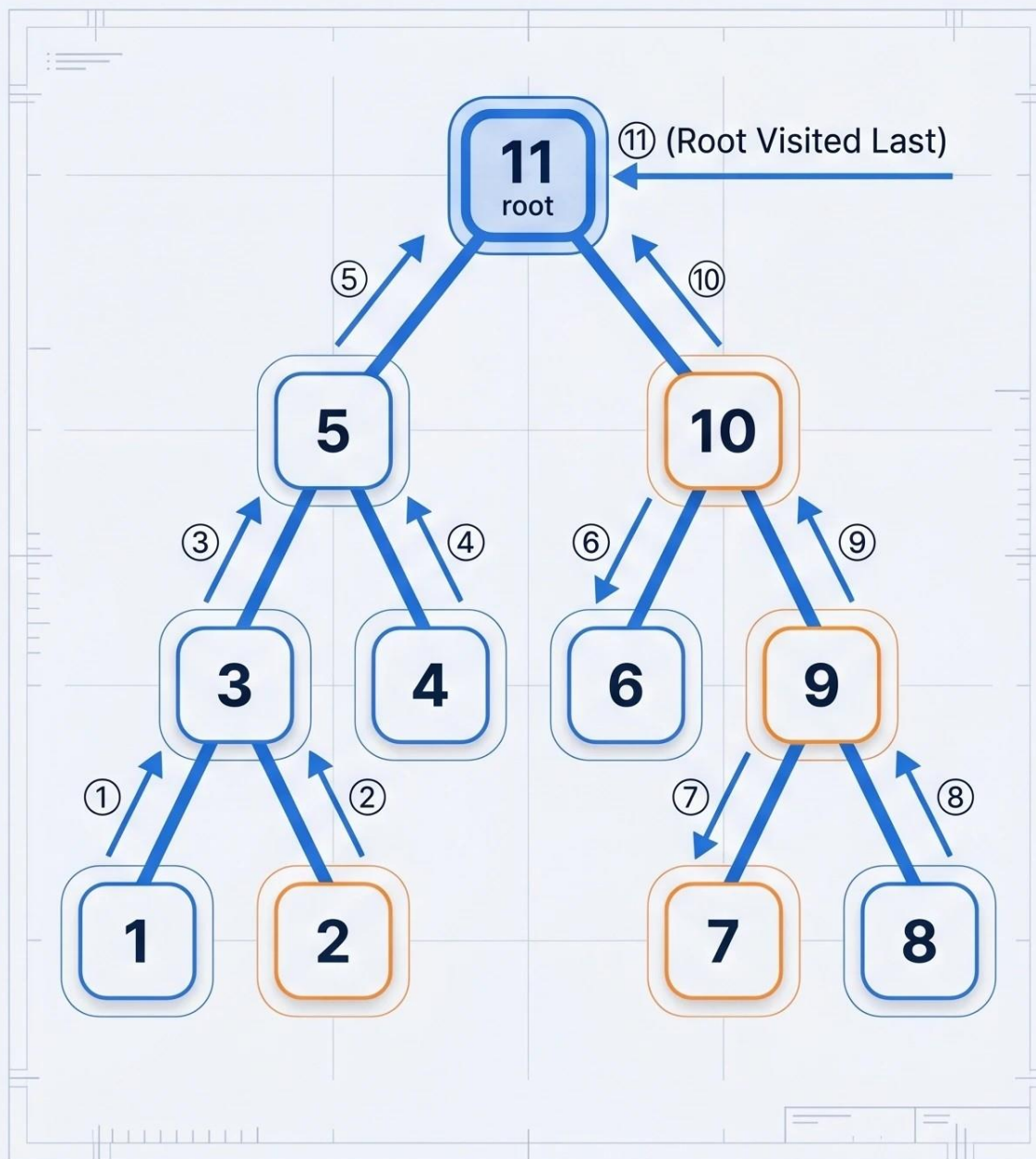


R: 오른쪽 서브트리 후위 순회  
(Right Subtree Post-order)

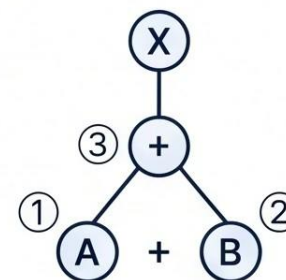


V: 루트 노드 마지막 방문  
(Visit Root Last)

절차:  
왼쪽 → 오른쪽 → 루트  
순 방문

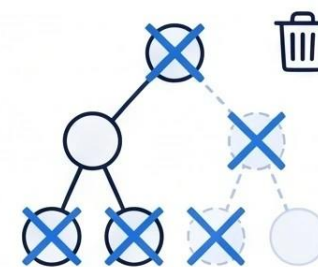


### 활용: 자식 먼저 처리



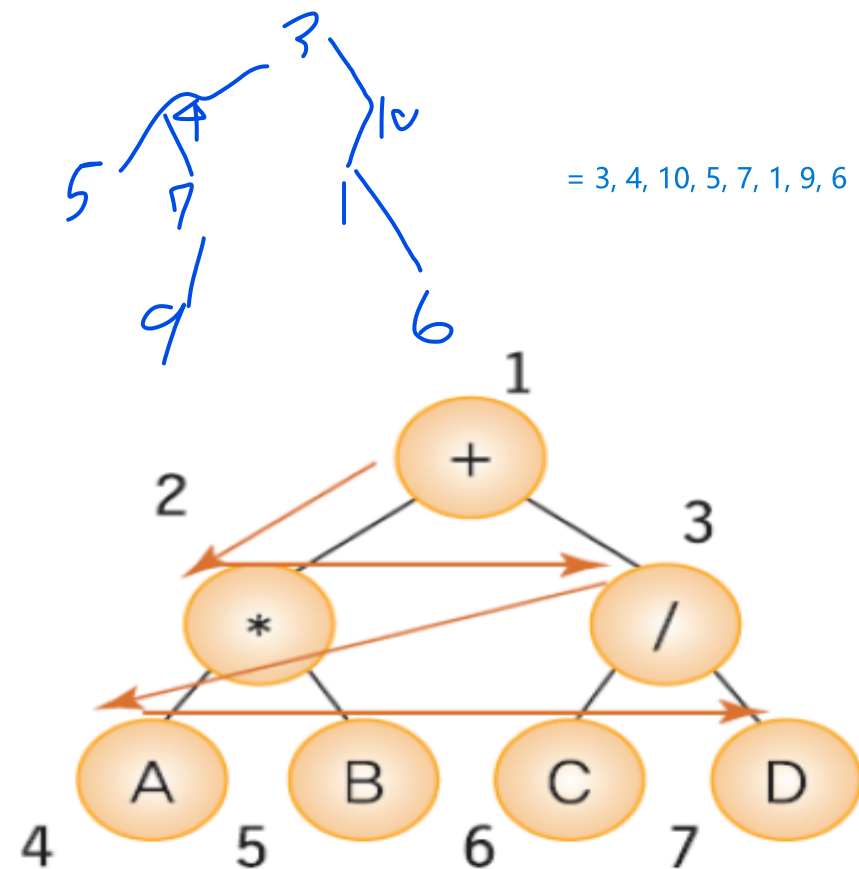
수식 트리 계산

### 활용: 메모리 관리

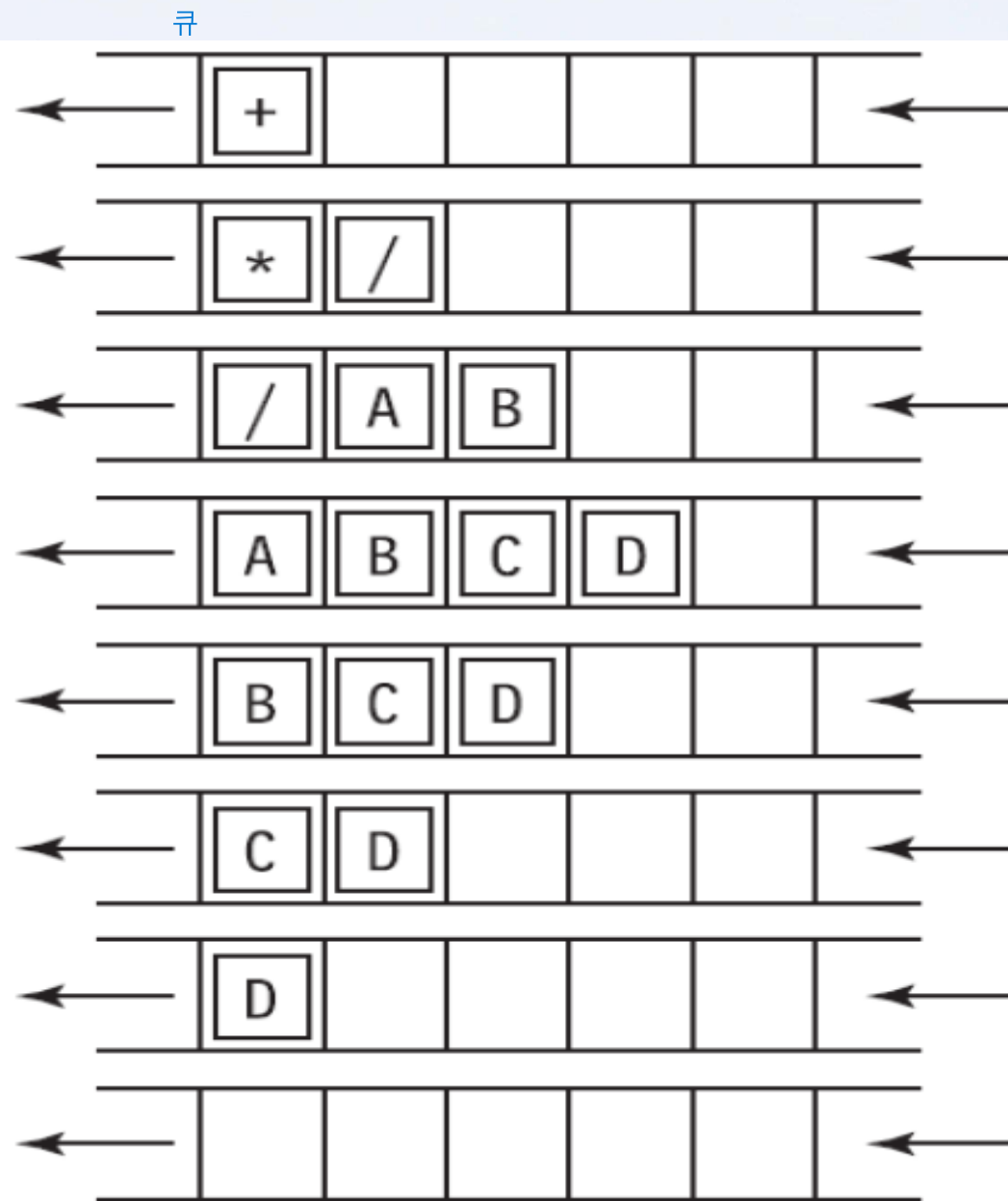


메모리 해제

# 레벨 순회 (Level-order Traversal)



종료 조건 = 큐가 비었을때

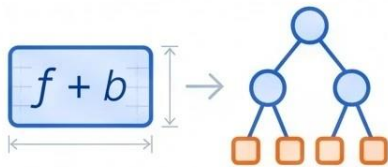




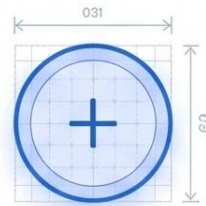
# 수식 트리 수식 트리 (Expression Trees) 핵심 개념

## 수식 트리 정의

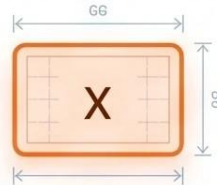
산술식(또는 논리식)을  
트리 형태로 표현한 구조.  
컴파일러는 수식을  
트리로 변환해  
최적화·코드 생성을  
수행합니다.



## 노드 구성



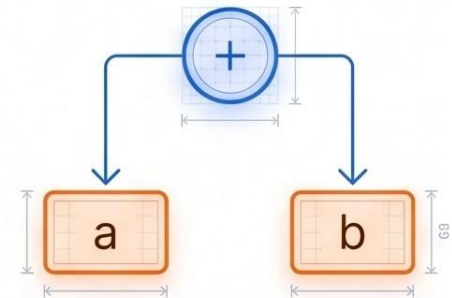
연산자 (Operator)  
[비단말노드]



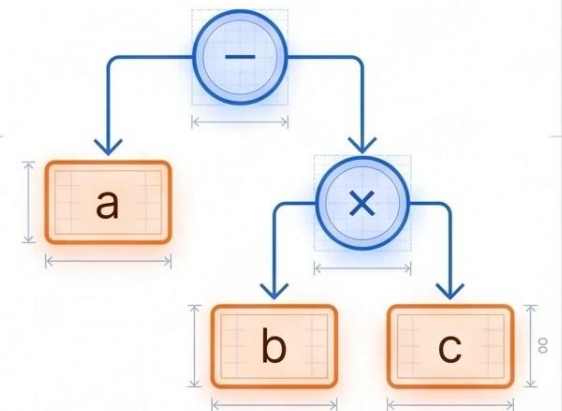
피연산자 (Operand)  
[단말노드]

## 트리 예시

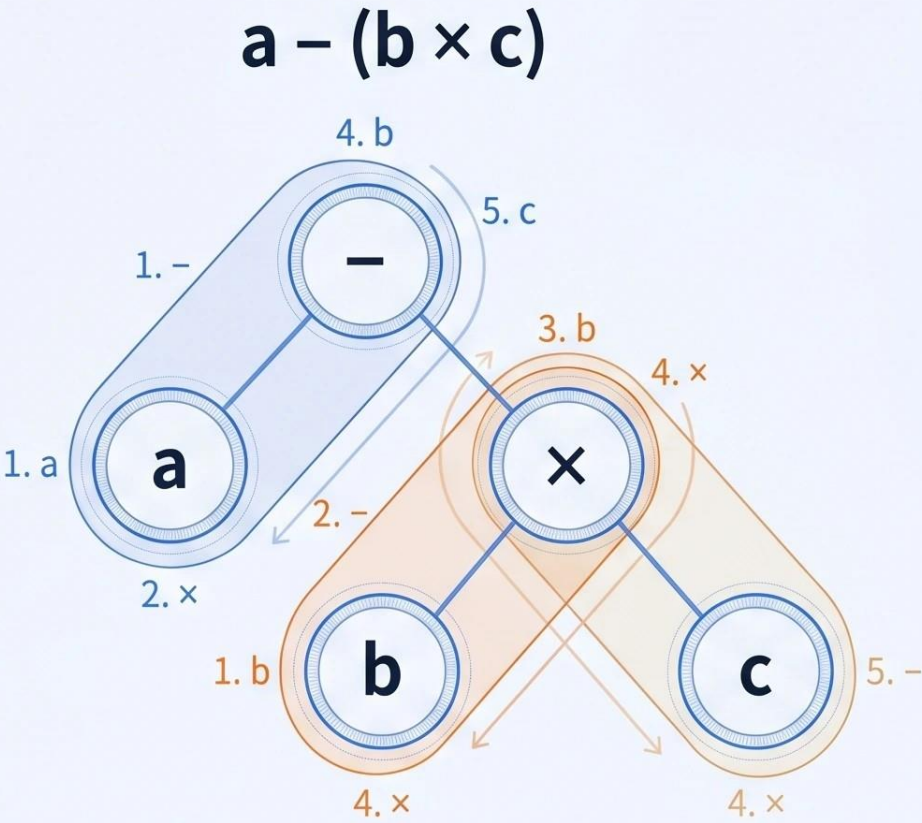
예시 1:  $a + b$



예시 2:  $a - (b \times c)$



# 수식 트리 순회와 표기법 (Expression Tree Traversal & Notations)



- 전위(Pre-order): VLR (Visit, Left, Right)
- 중위(In-order): LVR (Left, Visit, Right)
- 후위(Post-order): LRV (Left, Right, Visit)

|                                    | 예시 1:<br>a + b | 예시 2:<br>a - (b × c) | 예시 3:<br>(a < b) or<br>(c < d) |
|------------------------------------|----------------|----------------------|--------------------------------|
| 전위 표기식<br>(Pre-order<br>Notation)  | + a b          | - a × b c            | or<br>< a b<br>< c d           |
| 중위 표기식<br>(In-order<br>Notation)   | a + b          | a - (b × c)          | (a < b)<br>or<br>(c < d)       |
| 후위 표기식<br>(Post-order<br>Notation) | a b +          | a b c × -            | a b < c d<br>< or              |

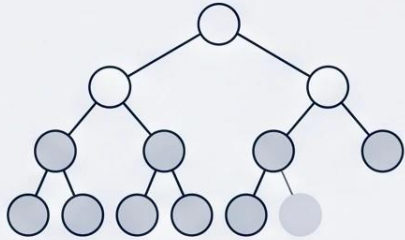
**보충:** 후위 표기는 스택 기반 계산에 적합하여 RPN 계산기와 컴파일러에 활용된다.  
(Note: Post-order notation is suitable for stack-based calculations and is used in RPN calculators and compilers.)



# 최대 힙 (Max Heap): 정의 및 구조

최소힙도 있음

최상단 노드 삭제되면, 그 아래 중에서 가장 큰 값을 다시 최상단으로



## 완전 이진 트리 구조

모든 레벨이 가득 차거나,  
마지막 레벨은 왼쪽부터 채워짐



정의: 완전 이진 트리  
+ 부모 키  $\geq$  자식 키

구조적 특징:  
우선순위 큐 구현에 사용

최소 힙  
(Min Heap)  
비교



부모 노드 키  $\leq$  자식 노드 키

구현 방식: 배열 기반 표현  
효율적인 인덱싱으로 트리 구조를 매핑

루트 노드 = 항상 최댓값



# 최대 힙 삽입

## 정의 (Definition)

최대 힙 삽입은 완전 이진 트리 형태와  
부모  $\geq$  자식 힙 속성을 유지하며  
노드 위치를 조정하는 과정

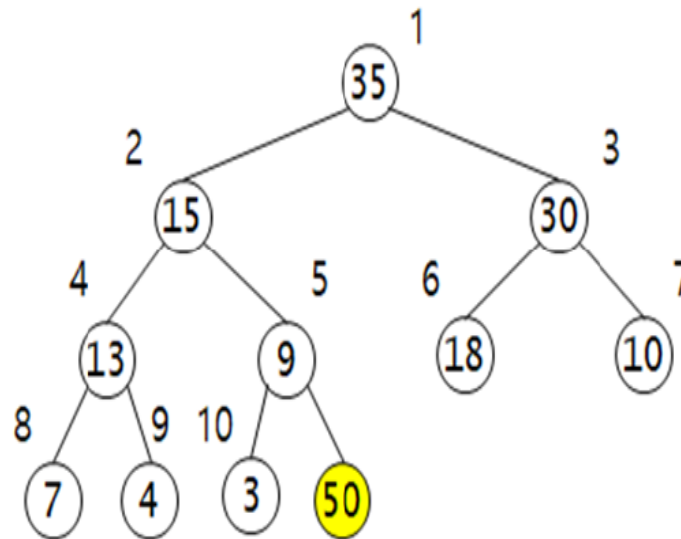
(1) 마지막 위치에 삽입

(2) 부모와 비교

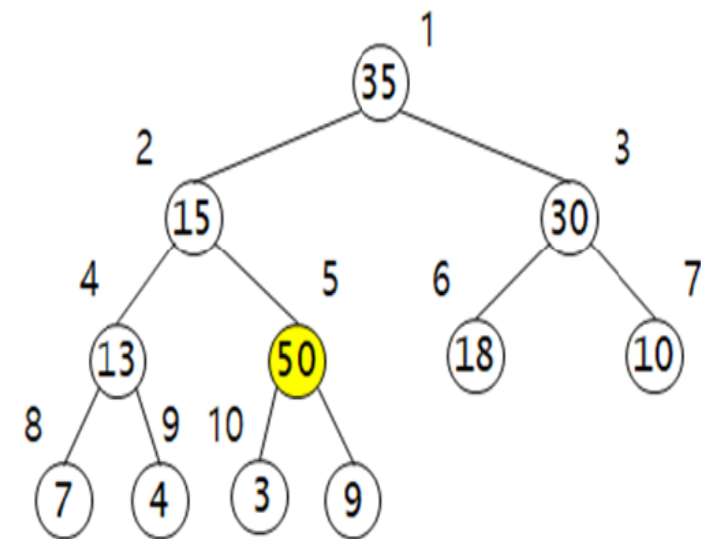
(3) 더 크면 swap

(4) 반복 (Repeat):  
루트 도달 또는 부모  $\geq$  자식  
만족 시까지 위로 이동

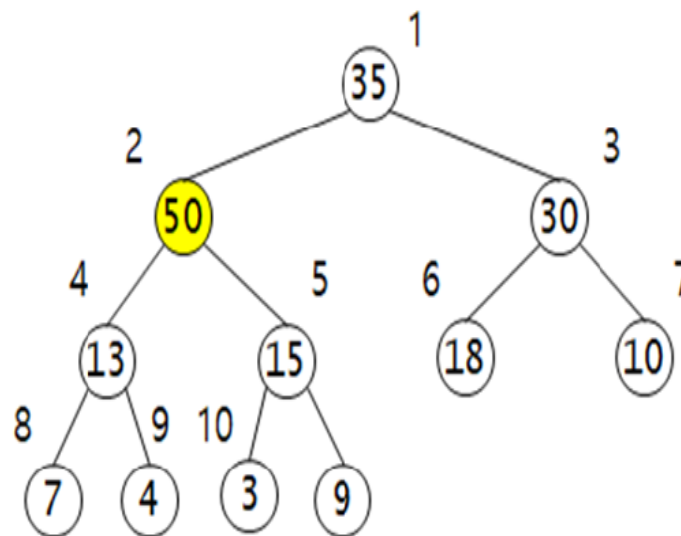
percolate up



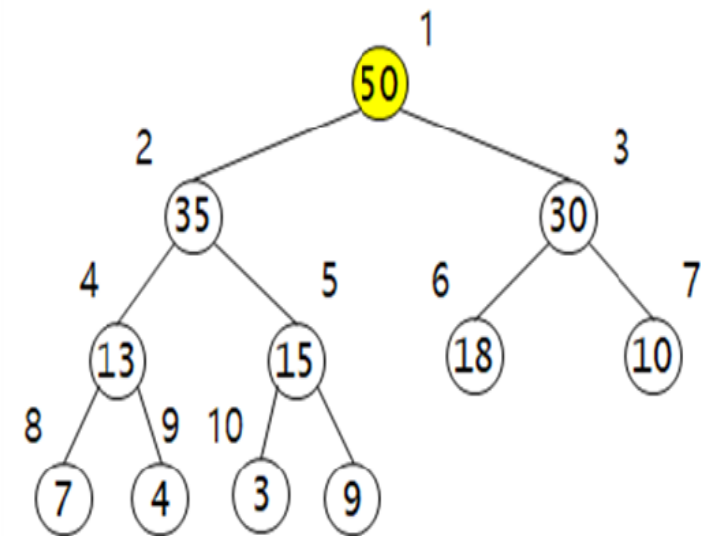
(a) 새로운 키 값 50의 삽입



(b) 부모 노드의 키 값 9와 교체



(c) 부모 노드의 키 값 15와 교체



(d) 부모 노드의 키 값 35와 교체

(a) 새로운 키 값 50을 인덱스 11에 삽입

|     |                                                                                   |    |    |    |    |   |    |    |   |   |    |    |
|-----|-----------------------------------------------------------------------------------|----|----|----|----|---|----|----|---|---|----|----|
| 인덱스 | 0                                                                                 | 1  | 2  | 3  | 4  | 5 | 6  | 7  | 8 | 9 | 10 | 11 |
| 값   |  | 35 | 15 | 30 | 13 | 9 | 18 | 10 | 7 | 4 | 3  | 50 |

(b) 새로운 키 값 50을 부모 노드의 값 9와 교체

|     |                                                                                   |    |    |    |    |    |    |    |   |   |    |    |
|-----|-----------------------------------------------------------------------------------|----|----|----|----|----|----|----|---|---|----|----|
| 인덱스 | 0                                                                                 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8 | 9 | 10 | 11 |
| 값   |  | 35 | 15 | 30 | 13 | 50 | 18 | 10 | 7 | 4 | 3  | 9  |

(c) 새로운 키 값 50을 부모 노드의 값 15와 교체

|     |                                                                                     |    |    |    |    |    |    |    |   |   |    |    |
|-----|-------------------------------------------------------------------------------------|----|----|----|----|----|----|----|---|---|----|----|
| 인덱스 | 0                                                                                   | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8 | 9 | 10 | 11 |
| 값   |  | 35 | 50 | 30 | 13 | 15 | 18 | 10 | 7 | 4 | 3  | 9  |

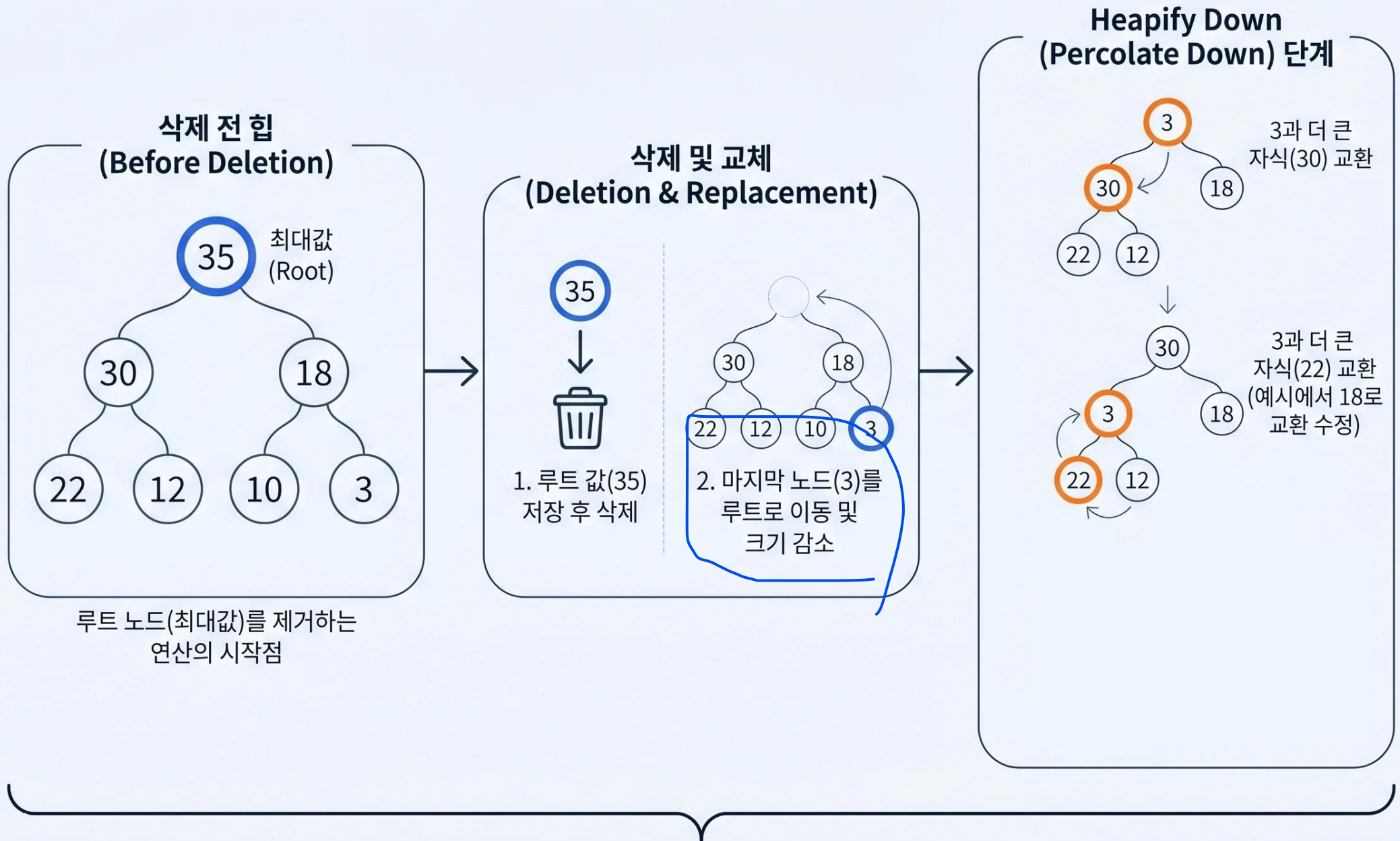
(d) 새로운 키 값 50을 부모 노드의 값 35와 교체

|     |                                                                                     |    |    |    |    |    |    |    |   |   |    |    |
|-----|-------------------------------------------------------------------------------------|----|----|----|----|----|----|----|---|---|----|----|
| 인덱스 | 0                                                                                   | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8 | 9 | 10 | 11 |
| 값   |  | 50 | 35 | 30 | 13 | 15 | 18 | 10 | 7 | 4 | 3  | 9  |

(그림 7.5) 배열로 표현된 최대 힙에서 새로운 키 값 50의 삽입 과정

# 최대 힙 삭제 연산: 루트 제거 및 구조 유지

(Max Heap Deletion: Root Removal and Structure Maintenance)



전체 과정: percolate down (heapify down)